

Digital Electronics and Microprocessor

Semester III — Detailed Unit-wise Notes (Course Code: 2418304)

Unit 1.0 — Number Systems, Boolean Algebra and Logic Gates

1.1 Different Number Systems

A **number system** is a method of representing numbers using a set of symbols and rules.

- bullet **Binary (base-2)**: uses digits 0 and 1; the native language of digital circuits.
- bullet **Octal (base-8)**: uses digits 0–7.
- bullet **Decimal (base-10)**: uses digits 0–9; the everyday number system.
- bullet **Hexadecimal (base-16)**: uses digits 0–9 and letters A–F; commonly used to represent memory addresses and binary data compactly.
- bullet **Conversion between number systems**: performed by repeated division (decimal to another base) or by grouping binary digits (binary $\leftrightarrow$ octal in groups of 3 bits, binary $\leftrightarrow$ hexadecimal in groups of 4 bits), and by positional weighted-sum expansion (any base to decimal).

1.2 Arithmetic Operations

Addition, subtraction, multiplication, and division can be performed directly in binary, octal, and hexadecimal using rules analogous to decimal arithmetic but with the respective base's carry/borrow rules (e.g., in binary, $1+1 = 10$, carry 1).

1.3 Complements

- bullet **1's Complement**: obtained by inverting every bit of a binary number ($0 \rightarrow 1$, $1 \rightarrow 0$).
- bullet **2's Complement**: obtained by adding 1 to the 1's complement of the number; widely used to represent negative numbers in digital systems because it allows subtraction to be performed using addition circuitry.

1.4 Data Representation

- bullet **Representation of negative numbers**: in 1's complement, the negative of a number is its bitwise inversion; in 2's complement, it is the inversion plus 1 — 2's complement is preferred since it has a single representation of zero and simplifies arithmetic circuits.
- bullet **Subtraction using complements**: to subtract B from A, add A to the complement of B and discard/handle the resulting carry — converts subtraction into addition, simplifying hardware design.

1.5 Boolean Algebra

Boolean Algebra is a mathematical system using two values (0 and 1, i.e., False/True) and the operators AND, OR, NOT, used to analyze and simplify digital logic circuits.

- bullet **Rules and laws**: Identity, Null, Idempotent, Complement, Commutative, Associative, Distributive, and Absorption laws.
- bullet **De Morgan's Theorem**: $\neg(A \cdot B) = \neg A + \neg B$ and $\neg(A + B) = \neg A \cdot \neg B$ — used to convert AND/OR expressions and simplify NAND/NOR-based circuits.

1.6 Standard Boolean Representation

- bullet **Sum of Products (SOP):** a Boolean expression written as the OR (sum) of AND (product) terms, e.g., $AB + A'C$.
- bullet **Product of Sums (POS):** a Boolean expression written as the AND (product) of OR (sum) terms, e.g., $(A+B)(A'+C)$.

1.7 Minimization

- bullet **Karnaugh Map (K-map):** a graphical method (grid of cells) used to simplify Boolean expressions by grouping adjacent 1s (or 0s), up to three (or more) variables — reduces the number of gates/terms needed.
- bullet **Simplification using Boolean laws:** algebraic manipulation using the laws above to reduce an expression to its minimal SOP/POS form before implementation.

1.8 Logic Gates and Applications

Logic gates are the basic building blocks of digital circuits, each with a symbol, truth table, and Boolean expression:

- bullet **AND:** output is 1 only if all inputs are 1. **OR:** output is 1 if any input is 1.
- bullet **NOT:** inverts the input. **Buffer:** passes input unchanged (used for signal amplification/isolation).
- bullet **NAND** (NOT-AND) and **NOR** (NOT-OR): universal gates — any logic function can be implemented using only NAND gates or only NOR gates.
- bullet **XOR (Exclusive-OR):** output is 1 only if inputs differ. **XNOR:** output is 1 only if inputs are the same.
- bullet Applications: arithmetic circuits, decision-making logic, error detection (XOR used in parity checking).

1.9 Implementation of Boolean Expressions Using Basic Gates

Any Boolean expression (in SOP or POS form) can be implemented directly using a combination of AND, OR, and NOT gates, with each product/sum term realized by one gate layer and the gates combined accordingly.

Unit 2.0 — Combinational and Sequential Logic Circuits

2.1 Arithmetic Circuits

- bullet **Half Adder:** a combinational circuit that adds two single bits (A, B) and produces a Sum and a Carry output; built using an XOR gate (sum) and an AND gate (carry).
- bullet **Full Adder:** adds two bits plus an incoming carry (A, B, Cin), producing Sum and Carry-out; used to build multi-bit adders by chaining (ripple-carry adder).
- bullet **Half Subtractor:** subtracts one bit from another, producing Difference and Borrow outputs.
- bullet **Full Subtractor:** subtracts one bit from another along with a borrow-in, producing Difference and Borrow-out.

2.2 Multiplexer

A **Multiplexer (MUX)** is a combinational circuit that selects one of several input signals and forwards it to a single output line, based on the value of select lines.

- bullet **2-to-1 MUX:** selects between 2 inputs using 1 select line.
- bullet **4-to-1 MUX:** selects between 4 inputs using 2 select lines.
- bullet **Applications:** data routing, communication systems (channel selection), implementing Boolean functions.

2.3 De-multiplexer

A **De-multiplexer (DEMUX)** takes a single input and routes it to one of several output lines, based on select line values — the reverse operation of a multiplexer.

- bullet **1-to-2 DEMUX:** routes input to 1 of 2 outputs using 1 select line.
- bullet **1-to-4 DEMUX:** routes input to 1 of 4 outputs using 2 select lines.
- bullet **Applications:** data distribution, signal demultiplexing in communication systems.

2.4 Encoder and Decoder

- bullet **Encoder:** a combinational circuit that converts 2^n input lines into an n-bit binary code (compresses information, e.g., octal-to-binary encoder).
- bullet **Decoder:** a combinational circuit that converts an n-bit binary code into 2^n output lines (expands information, e.g., used to select memory chips or activate one of many output devices).

2.5 Flip-Flops

A **flip-flop** is a basic sequential circuit (memory element) that stores one bit of data, with the output depending on present inputs and the previous state.

- bullet **SR Flip-Flop:** has Set (S) and Reset (R) inputs; S=1 sets output to 1, R=1 resets output to 0; S=R=1 is an invalid/undefined state.
- bullet **JK Flip-Flop:** an improvement over SR that eliminates the invalid state — when J=K=1, the output toggles.
- bullet **T (Toggle) Flip-Flop:** output toggles on every clock pulse when T=1; derived from JK by tying J and K together.

- bullet **D (Data/Delay) Flip-Flop**: output simply follows the D input at the clock edge, used to "store" a bit.
- bullet **Master-Slave JK Flip-Flop**: two JK flip-flops cascaded (master controlled by clock, slave by inverted clock) to eliminate race-around problems found in simple JK flip-flops.

2.6 Shift Registers

A **shift register** is a sequential circuit made of cascaded flip-flops that shifts stored bits by one position on each clock pulse.

- bullet **Serial In Serial Out (SISO)**: data enters and exits one bit at a time, serially.
- bullet **Serial In Parallel Out (SIPO)**: data enters serially, but all bits are available simultaneously at parallel outputs.
- bullet **Parallel In Serial Out (PISO)**: data is loaded in parallel, but shifted out serially.
- bullet **Parallel In Parallel Out (PIPO)**: data is loaded and read out entirely in parallel.

2.7 Counters

A **counter** is a sequential circuit that goes through a predefined sequence of states (counts) in response to clock pulses.

- bullet **Modulus of counter**: the number of distinct states (the count length) before the counter repeats, e.g., a mod-8 counter cycles through 8 states.
- bullet **Asynchronous (Ripple) Counter**: flip-flops are not all triggered by the same clock signal simultaneously; the output of one flip-flop drives the clock of the next, causing a "ripple" delay — can count up or down.
- bullet **Synchronous Counter (Ring Counter)**: all flip-flops are triggered by the same clock signal simultaneously; a **ring counter** shifts a single 1 (or 0) bit circularly through a loop of flip-flops, used for sequencing/timing applications.

Unit 3.0 — Data Converters and Memory Devices

3.1 Data Converters

- bullet **Op-Amp (Operational Amplifier):** a high-gain electronic amplifier used as a basic building block in analog circuits.
- bullet **Inverting configuration:** output is 180° out of phase with the input, with gain = $-R_f/R_{in}$.
- bullet **Non-inverting configuration:** output is in phase with the input, with gain = $1 + R_f/R_{in}$.
- bullet **Digital-to-Analog Converter (DAC):** converts a digital (binary) signal into a corresponding analog voltage/current — used in audio playback, signal generation.
- bullet **Analog-to-Digital Converter (ADC):** converts a continuous analog signal into a discrete digital representation — used in sensors, digital measurement instruments.

3.2 Random Access Memory (RAM)

RAM is volatile memory used for temporary storage during program execution, allowing read and write access.

- bullet **Static RAM (SRAM):** stores data using flip-flop-like latching circuits; faster but more expensive and uses more power per bit; does not need refreshing.
- bullet **Dynamic RAM (DRAM):** stores data as charge in a capacitor; requires periodic refreshing to retain data; cheaper and denser, used as main memory.

3.3 Read Only Memory (ROM)

ROM is non-volatile memory that retains its contents even when power is off, primarily used for permanent or semi-permanent storage of firmware/instructions.

- bullet **PROM (Programmable ROM):** can be programmed once by the user after manufacturing.
- bullet **EPROM (Erasable PROM):** can be erased (using ultraviolet light) and reprogrammed multiple times.
- bullet **EEPROM (Electrically Erasable PROM):** can be erased and reprogrammed electrically, without removing it from the circuit.

Unit 4.0 — Basics, Instruction Set and Programming of 8085 Microprocessor

4.1 Basics of Microprocessor

- Evolution of Microprocessors:** progressed from 4-bit (Intel 4004) to 8-bit (8085), 16-bit, 32-bit, and modern 64-bit multi-core processors, with increasing speed, integration, and capability.
- Architecture of 8085:** an 8-bit microprocessor consisting of the ALU (Arithmetic Logic Unit), general-purpose registers (B, C, D, E, H, L), special registers (Accumulator, Program Counter, Stack Pointer, Flag register), Instruction Decoder, and Timing & Control unit — connected to address bus (16-bit), data bus (8-bit), and control bus.
- Pin diagram:** 40-pin IC with address lines (A8–A15), address/data lines (AD0–AD7, multiplexed), control signals (RD, WR, ALE), interrupt pins (INTR, INTA, RST), and power/clock pins.
- Timing Diagram and Memory Organization:** shows the sequence of signals (address, data, control) over time for instruction fetch/execute cycles; memory is organized as a linear array of bytes addressed 0000H to FFFFH.
- Interrupts:** signals that temporarily halt normal program execution to service an urgent event — 8085 supports hardware interrupts (TRAP, RST5.5, RST6.5, RST7.5, INTR) and software interrupts (via RST instructions).

4.2 Instruction Set

The 8085 instruction set is grouped into:

- Data Transfer Instructions:** move data between registers/memory (e.g., MOV, MVI, LDA, STA).
- Arithmetic Instructions:** perform addition, subtraction, increment, decrement (e.g., ADD, SUB, INR, DCR).
- Logical Instructions:** perform bitwise operations and comparisons (e.g., ANA, ORA, XRA, CMP).
- Branching Instructions:** alter the normal sequential flow of execution (e.g., JMP, JZ, JNZ, CALL, RET).
- Control Instructions:** control processor operation (e.g., HLT, NOP, EI, DI).

4.3 Addressing Modes

Addressing mode specifies how the operand of an instruction is located:

- Immediate Addressing:** the operand is given directly within the instruction (e.g., MVI A, 05H).
- Register Addressing:** the operand is in a specified CPU register (e.g., MOV A, B).
- Direct Addressing:** the instruction specifies the memory address of the operand directly (e.g., LDA 2000H).
- Indirect Addressing:** the instruction specifies a register pair that holds the address of the operand (e.g., MOV A, M, where M refers to the memory location pointed to by HL).
- Indexed Addressing:** uses an index/base register plus an offset to compute the effective address (used in more advanced processors, conceptually extended from 8085's HL-based indirect addressing).

4.4 Assembly Language Programming

Assembly Language Programs (ALP) are written using mnemonics corresponding to machine instructions, assembled into machine code for execution. Typical ALPs handle tasks such as arithmetic operations on multi-byte numbers, data movement, sorting, and array processing, using the instruction set and addressing modes described above.

Unit 5.0 — Interfacing with 8085 Microprocessor

5.1 Programmable Peripheral Interface (PPI) — Intel 8255

The **8255 PPI** is a programmable parallel I/O interfacing chip that provides three 8-bit ports (Port A, Port B, Port C), used to generate I/O ports for connecting external devices (LEDs, switches, sensors) to the 8085.

- Ports can be configured in different operating **modes**: Mode 0 (Simple I/O), Mode 1 (Strobed I/O with handshaking), and Mode 2 (Bidirectional, only for Port A) — configured via a control word written to the chip's control register.

5.2 Programmable Interval Timers (Intel 8253/8254)

The **8253/8254** is a programmable timer/counter chip with three independent 16-bit counters, used for generating precise time delays, event counting, and waveform generation (e.g., generating periodic interrupts, real-time clock functions) in microprocessor-based systems.

5.3 Overview of Memory Chips and Their Interfaces

Memory chips (RAM/ROM ICs) are interfaced to the 8085 by connecting their address, data, and control (chip-select, read, write) lines to the corresponding microprocessor bus lines, with **address decoding circuits** (e.g., using decoders like 74138) used to select the correct chip for a given address range.

5.4 Overview of Other Interfacing Chips

- Other commonly used interfacing chips include the **8259** (Programmable Interrupt Controller — manages multiple interrupt sources and priorities) and the **8251** (Programmable Communication Interface/USART — handles serial data communication).
- Serial vs Parallel communication:**
- Serial communication** transmits data one bit at a time over a single line — slower but requires fewer wires, suited for long-distance communication (e.g., UART, RS-232).
- Parallel communication** transmits multiple bits simultaneously over multiple lines — faster but requires more wiring, suited for short-distance, high-speed transfer (e.g., between the microprocessor and nearby memory/I/O chips).